UI model often considered more flexible than Angular or Vue?"

Analysis: This interview question targets Angular and Vue developers transitioning to React or candidates familiar with multiple frontend frameworks. It assesses their ability to compare React's unopinionated, composable architecture with the more structured approaches of Angular and Vue.

Answer: React's component-based UI model is considered more flexible because it treats components as plain JavaScript functions or classes that return UI, without enforcing rigid structures or templating syntax. Unlike Angular, which uses declarative templates within a highly opinionated framework, and Vue, which separates logic, templates, and styles into `.vue` files, React allows co-locating markup, logic, and styles in JSX. This enables highly reusable, composable, and dynamic components.

React's unopinionated design gives developers the freedom to choose their own state management, routing, and styling solutions, while Angular and Vue offer more builtin conventions but less flexibility for highly customized architectures.

For example, building a dynamic dashboard where charts, tables, and widgets are rendered conditionally and updated independently is often simpler in React. Its one-way data flow and Hooks API allow fine-grained control over lifecycles and state.

While Angular's two-way data binding and Vue's directives are powerful, they can introduce debugging challenges in large apps. React's emphasis on functional programming and composability makes it ideal for complex UIs with deeply nested, reusable components.

Initial question (fundamental skill): "Can browsers understand JSX directly?"

Answer: No, browsers cannot parse JSX directly. JSX must be transpiled by tools such as Babel or esbuild into plain JavaScript—usually calls to React's `createElement` function—that browsers can execute. This transpilation enables developers to write clear, HTML-like syntax while producing